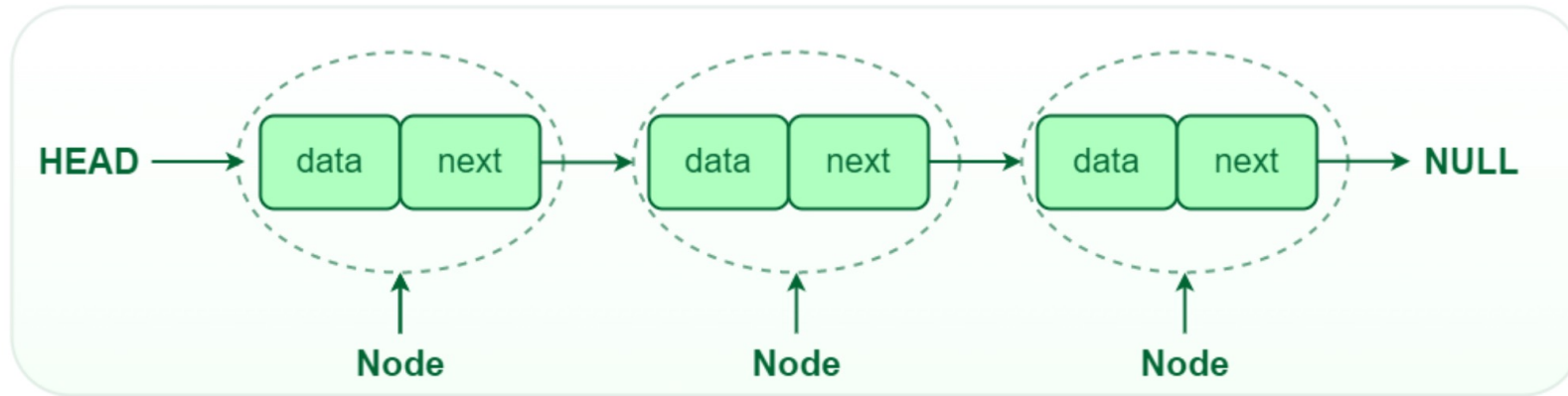


Implement the linked list abstract data type  
for floating point number

# What a linked list looks like?



```
// A Single linked list node
typedef struct _node {
    double value;           /* value stored in the node */
    struct _node * next;    /* pointer to the next node */
} Node;
```

A linked list is a linear data structure, in which the elements are not stored at contiguous memory locations.

# What a linked list looks like?

An example:

```
Node *n1 = malloc(sizeof(Node)); Node *n2 = malloc(sizeof(Node)); //
```

define 2 nodes

```
n1->value = 3; n1->next = n2; n2->value = 2; n2->next = NULL; //
```

define a linked list with value [3, 2]

n1 is the head of the linked list

# The APIs we need to implement

```
void list_init_empty(Node **list); // Initialize the linked list as an empty list
void list_init(Node **list, double *arr, int n); // Initialize the linked list with an
array of n elements
Node* list_insert(Node *list, double value); // Insert value to the front of the linked
list
Node* list_delete(Node *list, double value); // Delete value from the linked list
int list_contain(Node *list, double value); // Check if value exists in the list or not
void list_free(Node **list); // Free the list, if list is not NULL
char* list_print(Node *list); // The print function print the list in an output string
```

# The implementations

```
// Initialize the linked list as an empty list  
// An empty list is defined as a null pointer
```

```
void list_init_empty(Node **list){  
    // Your code here  
    *list = NULL;  
}
```

# The implementations

```
// Initialize the linked list with an array of n elements  
// The last element of the arr is the first element in the linked list
```

```
void list_init(Node **list, double *arr, int n){  
    // Your code here  
    int i;  
    *list = NULL;  
    for (i = 0; i < n; i++) {  
        *list = list_insert(*list, arr[i]);  
    }  
}
```

# The implementations

```
// Insert value to the front of the linked list  
// Return the head of the linked list
```

```
Node* list_insert(Node *list, double value){  
    // Your code here  
    Node *tmpnode = malloc(sizeof(Node));  
    tmpnode->value = value;  
    tmpnode->next = list;  
  
    return tmpnode;  
}
```

# The implementations

// Delete value from the linked list  
// If value (could be multiple values) is in the list, then delete it, or do nothing.

```
Node* list_delete(Node *list, double value){  
    // Your code here  
    Node *ptr = list, *prev = NULL;  
    Node *to_be_free;  
  
    while (ptr != NULL) {  
        if (fabs(ptr->value - value) < 1e-8) {  
            to_be_free = ptr;  
            ptr = ptr->next;  
            if (prev == NULL) {  
                free(to_be_free);  
                list = ptr;  
            } else {  
                free(to_be_free);  
                prev->next = ptr;  
            }  
            continue;  
        }  
        prev = ptr;  
        ptr = ptr->next;  
    }  
    return list;  
}
```

# The implementations

```
// Check if value exists in the list or not  
// If value is in the list, return 1, otherwise 0
```

```
int list_contain(Node *list, double value){  
    // Your code here  
    Node *ptr = list;  
  
    while (ptr != NULL) {  
        if (fabs(ptr->value - value) < 1e-8)  
            return 1;  
        ptr = ptr->next;  
    }  
  
    return 0;  
}
```

# The implementations

```
// Free the list, if list is not NULL  
// Assign NULL to the *list
```

```
void list_free(Node **list){  
    // Your code here  
    Node *ptr = *list, *prev = NULL;  
    while (ptr != NULL) {  
        prev = ptr;  
        ptr = ptr->next;  
        free(prev);  
    }  
    *list = NULL;  
}
```

# The implementations

```
// The print function print the list in an output string  
// Example: "14.00 15.00 17.50"
```

```
char* list_print(Node *list){  
    // Your code here  
    int max_len = 7*100+2; // rough approximation  
    char *output = malloc(sizeof(char)*max_len);  
  
    memset(output, 0, sizeof(char)*max_len);  
  
    Node *ptr = list;  
    while (ptr != NULL) {  
        sprintf(output, "%s%.2f ", output, ptr->value);  
        ptr = ptr->next;  
    }  
    return output;  
}
```